반도체장비데이터관리 07

SQL: DML, LEFT JOIN, DDL

김 민 진 | muea.mj@gmail.com

DML: 데이터 조작어

- 삽입, 수정, 삭제

데이터 조작어(DML : Data Manipulation Language)

명령어의 종류	명령어	설명
데이터 조작어	SELECT	데이터베이스에 들어 있는 데이터를 조회하거나 검색하기 위한 명령어
	INSERT UPDATE DELETE	데이터베이스의 테이블에 들어 있는 데이터에 변형을 가하는 종류의 명령어 예) 테이블에 새로운 행을 추가, 원하지 않는 데이터를 삭제하거나 수정

https://yunamom.tistory.com/318

INSERT 문

- 테이블에 새로운 투플을 삽입하는 명령3

- 기본 구조:

```
INSERT INTO 테이블이름[(속성리스트)]
        VALUES (값리스트);
```

- 예시:

```
INSERT INTO Book
        VALUES (12, '스포츠 의학', '한솔의학서적', 90000);
```

- 새로운 투플을 삽입할 때 속성 이름을 생략 가능
- 단, 데이터의 입력 순서는 속성의 순서와 일치해야 함

```
INSERT INTO Book(bookid, bookname, price, publisher)
        VALUES (13, '스포츠 의학', 90000, '한솔의학서적');
```

- 데이터는 항상 속성의 순서대로 입력하지 않아도 됨
- 몇 개의 속성만 입력해야 한다면 해당하는 속성만 명시하면 됨

INSERT

질의 3-44

Book 테이블에 새로운 도서 '스포츠 의학'을 삽입하시오. 스포츠 의학은 한솔의학서적에서 출간했으며 가격은 90,000원이다.

```
INSERT INTO Book(bookid, bookname, publisher, price)
       VALUES (11, '스포츠 의학', '한솔의학서적', 90000);
```

bookid	bookname	publisher	price
1	축구의 역사	굿스포츠	7000
2	축구 아는 여자	나무수	13000
3	축구의 이해	대한미디어	22000
4	골프 바이블	대한미디어	35000
5	피겨 교본	굿스포츠	8000
6	배구 단계별기술	굿스포츠	6000
7	야구의 추억	이상미디어	20000
8	야구를 부탁해	이상미디어	13000
9	올림픽 이야기	삼성당	7500
10	Olympic Champions	Pearson	13000
11	스포츠 의학	한솔의학서적	90000

INSERT

질의 3-45

Book 테이블에 새로운 도서 '스포츠 의학'을 삽입하시오. 스포츠 의학은 한솔의학서적에서 출간했으며 가격은 미정이다.

```
INSERT INTO Book(bookid, bookname, publisher)
       VALUES (14, '스포츠 의학', '한솔의학서적');
```

bookid	bookname	publisher	price
1	축구의 역사	굿스포츠	7000
2	축구 아는 여자	나무수	13000
3	축구의 이해	대한미디어	22000
4	골프 바이블	대한미디어	35000
5	피겨 교본	굿스포츠	8000
6	배구 단계별기술	굿스포츠	6000
7	야구의 추억	이상미디어	20000
8	야구를 부탁해	이상미디어	13000
9	올림픽 이야기	삼성당	7500
10	Olympic Champions	Pearson	13000
11	스포츠 의학	한솔의학서적	90000
12	스포츠 의학	한솔의학서적	90000
13	스포츠 의학	한솔의학서적	90000
14	스포츠 의학	한솔의학서적	NULL

UPDATE 문

- 특정 속성값을 수정하는 명령

- 기본 구조:

```
UPDATE   테이블이름
SET      속성이름 1=값 1[, 속성이름 2=값 2, ...]
[WHERE   <검색조건>];
```

UPDATE

질의 3-47

Customer 테이블에서 고객번호가 5인 고객의 주소를 '대한민국 부산'으로 변경하시오.

```
SET SQL_SAFE_UPDATES=0;   /* Safe Updates 옵션 미 해제 시 실행 */
UPDATE   Customer
SET      address ='대한민국 부산'
WHERE    custid=5;
```

custid	name	address	phone
1	박지성	영국 맨체스터	000-5000-0001
2	김연아	대한민국 서울	000-6000-0001
3	김연경	대한민국 경기도	000-7000-0001
4	추신수	미국 클리블랜드	000-8000-0001
5	박세리	대한민국 부산	NULL

DELETE 문

- 테이블에 있는 기존 투플을 삭제하는 명령

- 기본 구조:

```
DELETE FROM 테이블이름
[WHERE 검색조건];
```

  - 〈검색조건〉에 해당되는 투플을 삭제
  - 〈검색조건〉이 없으면 모든 투플을 삭제

DELETE

질의 3-49

Book 테이블에서 도서번호가 11인 도서를 삭제하시오.

```
DELETE FROM    Book
WHERE  bookid = 11;
```

bookid	bookname	publisher	price
1	축구의 역사	굿스포츠	7000
2	축구 아는 여자	나무수	13000
3	축구의 이해	대한미디어	22000
4	골프 바이블	대한미디어	35000
5	피겨 교본	굿스포츠	8000
6	배구 단계별기술	굿스포츠	6000
7	야구의 추억	이상미디어	20000
8	야구를 부탁해	이상미디어	13000
9	올림픽 이야기	삼성당	7500
10	Olympic Champions	Pearson	13000
12	스포츠 의학	한솔의학서적	90000
13	스포츠 의학	한솔의학서적	90000
14	스포츠 의학	Pearson	NULL
21	Zen Golf	Pearson	12000
22	Soccer Skills	Human Kinetics	15000

DELETE

질의 3-50

모든 고객을 삭제하시오.

```
DELETE   FROM    Customer;
```

```
Error Code: 1451. Cannot delete or update a parent row: a foreign key constraint fails
(`madangdb`.`orders`, CONSTRAINT `orders_ibfk_1` FOREIGN KEY (`custid`) REFERENCES
`customer`
```

- 이 SQL 문은 실행되지 않음
- Orders 테이블에서 Customer.custid 속성을 외래키로 참조하고 있기 때문
- 제약이 해제되지 않으면 데이터 삭제가 중지됨

INNER JOIN vs LEFT JOIN

카테시안 곱

- *JOIN에 ON 조건이 없으면,* 카테시안 곱 연산과 동일하다.
- 카테시안 곱은 왼쪽 테이블의 각 행마다 오른쪽 테이블의 모든 행을 붙인다

```
SELECT *
FROM book
JOIN orders;
```

```
SELECT *
FROM orders
JOIN book;
```

- 예시
  - book (3행), orders (4행)이라면,
    book × orders → 12행
    orders × book → 12행
  - 행 개수는 동일하고, 결과에서 컬럼의 순서만 바뀐다. → 거의 차이가 없다.

카테시안 곱

- 예시

Equipment (장비)

equipment_id	model_name
1	A
2	B
3	C

UsageLog (사용기록)

id	equipment_id
1	1
2	1
3	2

Equipment × UsageLog

```
SELECT *
FROM usagelog
JOIN equipment
```

id	equipment_id	equipment_id	model_name
1	1	1	A
1	1	2	B
1	1	3	C
2	1	1	A
2	1	2	B
2	1	3	C
3	2	1	A
3	2	2	B
3	2	3	C

INNER JOIN

- **두 테이블에 모두 존재하는 데이터만 조회**
  - 연결 조건을 만족하는 행만 남고, 매칭 안 되는 데이터는 **자동으로 제거됨**

Equipment (장비)

equipment_id	model_name
1	A
2	B
3	C

UsageLog (사용기록)

id	equipment_id
1	1
2	1
3	2

```
SELECT *
FROM usagelog
JOIN equipment
ON usagelog.equipment_id = equipment.equipment_id;
```

id	equipment_id	equipment_id	model_name
1	1	1	A
2	1	1	A
3	2	2	B

- UsageLog에 있는 equipment_id와 Equipment에 있는 equipment_id가 **같은 것만 매칭**
- UsageLog에 없는 장비는 **애초에 매칭이 안 됨**
  → 결과에서 사라짐

한 번도 사용된 적이 없는 장비는 어떻게 알 수 있을까?

LEFT JOIN

- 왼쪽 테이블(From절)은 전부 유지 + 오른쪽은 매칭되는 것만 붙임
  - 왼쪽 기준으로 결과 생성하며 매칭 안 되는 경우 → NULL로 채워짐

Equipment (장비)

equipment_id	model_name
1	A
2	B
3	C

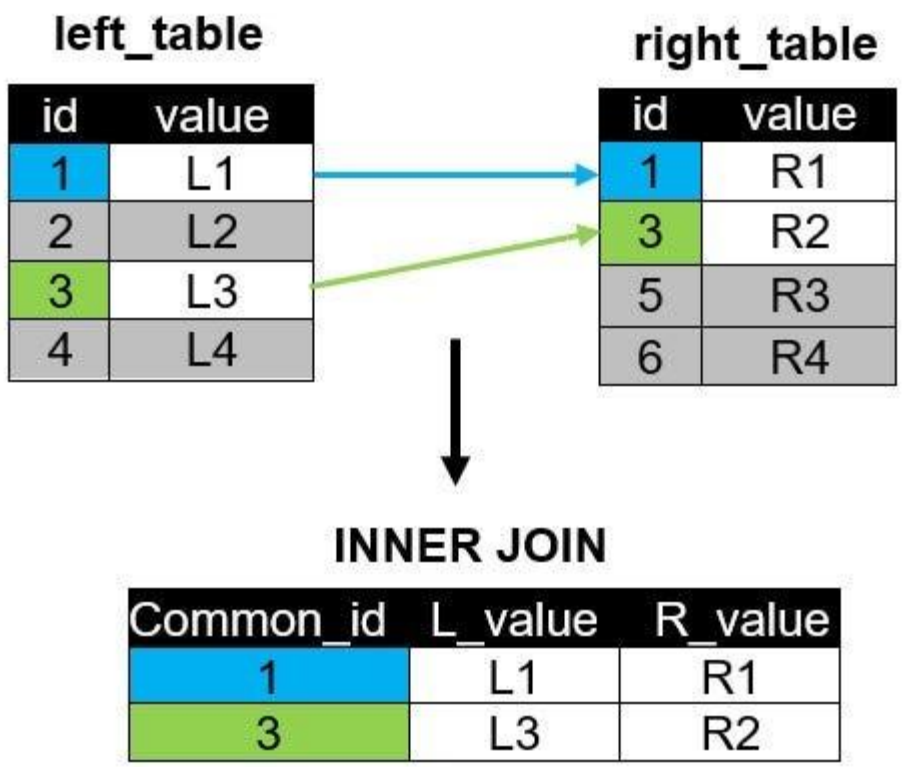
UsageLog (사용기록)

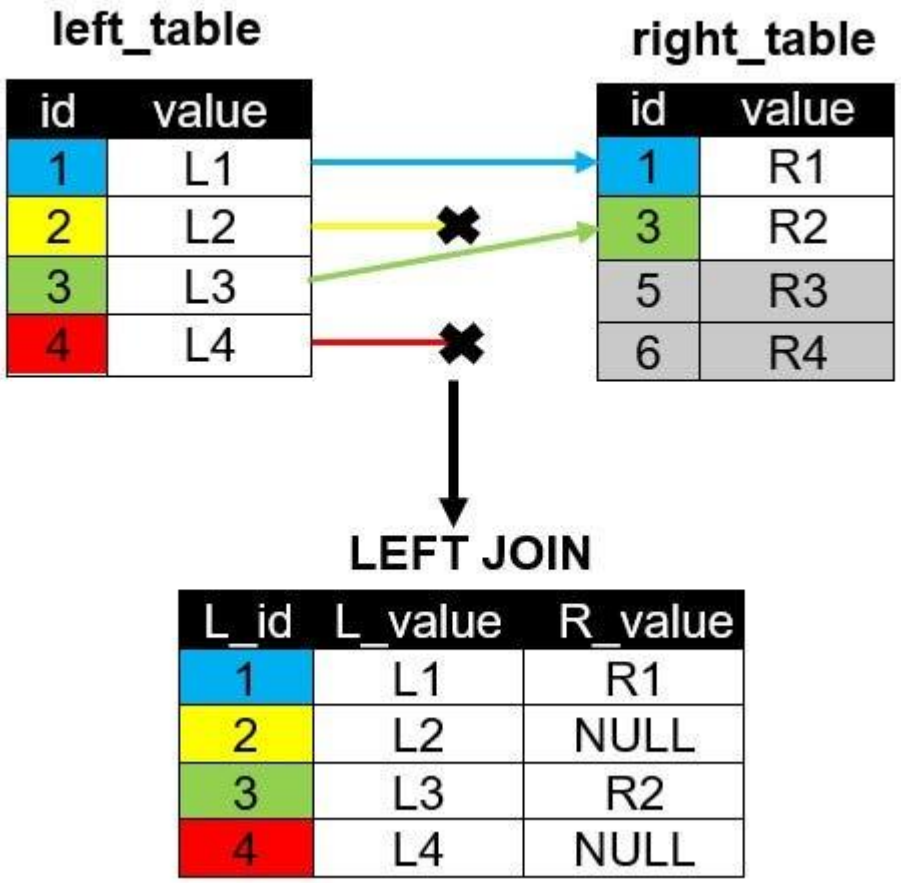
id	equipment_id
1	1
2	1
3	2

```sql
SELECT *
FROM Equipment
LEFT JOIN UsageLog
ON Equipment.equipment_id = UsageLog.equipment_id;
```

equipment_id	model_name	id	equipment_id
1	A	1	1
1	A	2	1
2	B	3	2
3	C	NULL	NULL

INNER JOIN과 LEFT JOIN





https://spardhax.medium.com/all-the-joins-in-sql-visualised-and-simplified-3df0687a8624

INNER JOIN과 LEFT JOIN

Table A

ID	Product
1	Chair
2	Table
3	Laptop
4	Phone
5	Flash Drive

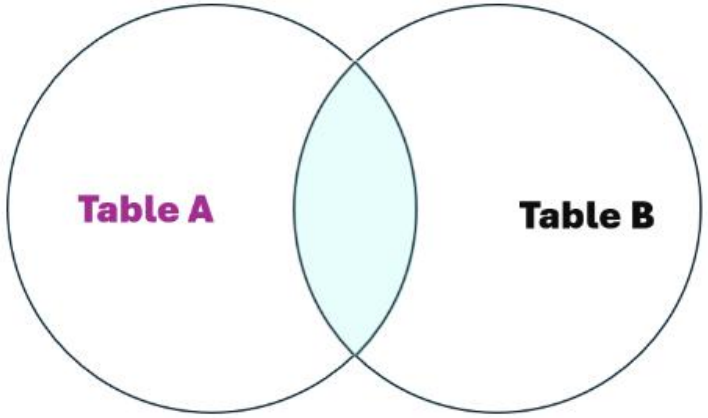


Table B

ID	Price
1	$80
2	$120
4	$799
6	$99

```
SELECT  *
FROM  Table A as a
INNER JOIN  Table B as b
ON a.ID = b.ID
```

↓

ID	Product	ID	Price
1	Chair	1	$80
2	Table	2	$120
4	Phone	4	$799

INNER JOIN과 LEFT JOIN

Table A

ID	Product
1	Chair
2	Table
3	Laptop
4	Phone
5	Flash Drive

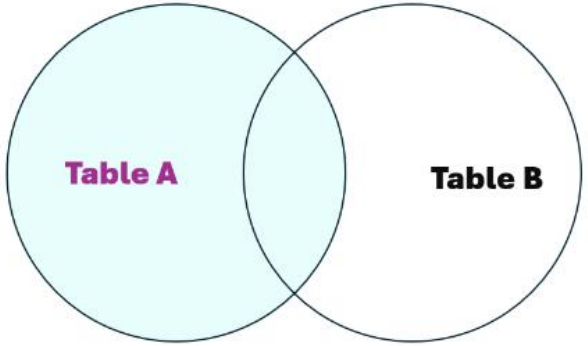


Table B

ID	Price
1	$80
2	$120
4	$799
6	$99

```
SELECT  *
FROM  Table A as a
LEFT JOIN  Table B as b
ON a.ID = b.ID
```

ID	Product	ID	Price
1	Chair	1	$80
2	Table	2	$120
3	Laptop	NULL	NULL
4	Phone	4	$799
5	Flash Drive	NULL	NULL

DDL: 데이터 정의어

데이터 정의어

❖데이터를 저장할 테이블 구조를 만드는 명령어

- CREATE : 테이블 구조를 만듦.
- ALTER : 이미 존재하는 테이블의 구조를 변경함.
- DROP : 테이블을 삭제함.

Data Type

❖데이터 형식 (Data Type)

- 특정 데이터를 받아들일 공간을 데이터의 유형별로 나누는 기준
- SQL 데이터 형식에는 크게 숫자, 문자, 날짜형이 있음
- 테이블 작성시 데이터 형식을 미리 설정해야 함
  - 설정한 형식이 아닌 다른 종류의 데이터가 들어오려고 하면 에러 발생

Data Type

- MySQL에서 사용하는 데이터 타입의 종류

표 3-11 데이터타입 종류

데이터 타입	설명	ANSI SQL 표준 타입
INTEGER INT	4바이트 정수형을 저장한다.	INTEGER, INT SMALLINT
NUMERIC(m,d) DECIMAL(m,d)	전체 자릿수 m, 소수점 이하 자릿수 d를 가진 숫자형을 저장한다.	DECIMAL(p, s) NUMERIC[(p,s)]
CHAR(n)	문자형 고정길이, 문자를 저장하고 남은 공간은 공백으로 채운다.	CHARACTER(n) CHAR(n)
VARCHAR(n)	문자형 가변길이를 저장한다.	CHARACTER VARYING(n) CHAR VARYING(n)
DATE	날짜형, 연도, 월, 일, 시간을 저장한다.	

정수형

- 소수점이 없는 숫자, 즉 인원수, 가격, 수량 등에 많이 사용
- 정수형의 크기와 범위 :

데이터 형식	바이트 수	숫자 범위
TINYINT	1	−128 ~ 127
SMALLINT	2	−32,768 ~ 32,767
INT	4	약 −21억 ~ + 21억
BIGINT	8	약 −900경 ~ +900경

정수형에는 TINYINT, SMALLINT, INT, BIGINT가 있습니다.

- Unsigned 키워드 : 값이 0부터 시작되는 정수
  - 예) TINYINT UNSIGNED는 0~255 범위의 값을 가질 수 있음

실수형

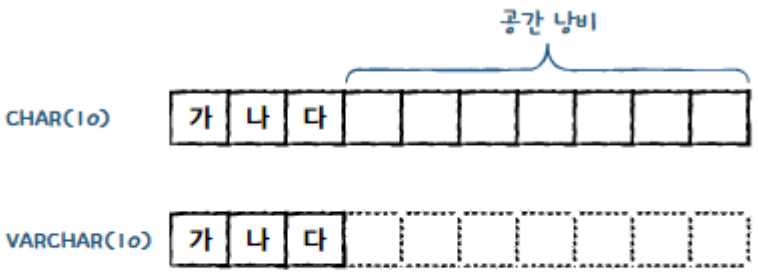
- 소숫점이 있는 숫자를 저장할 때 사용.

데이터 형식	바이트 수	설명
FLOAT	4	소수점 아래 7자리까지 표현
DOUBLE	8	소수점 아래 15자리까지 표현

문자형

- 글자를 저장하기 위해 사용.
- 입력할 최대 글자의 개수를 지정.

데이터 형식	바이트 수
CHAR(개수)	1~255
VARCHAR(개수)	1~16383



note CHAR, VARCHAR로만 쓰면 CHAR(1), VACHAR(1)과 동일합니다.

대량의 데이터 형식

- CHAR이나 VARCHAR로 저장할 수 없을 만큼 큰 데이터에 사용.
- BLOB (Binary Long Object)
  - 글자가 아닌 이미지, 동영상 등의 데이터 == 이진데이터
  - 테이블에 사진이나 동영상 같은 것을 저장하고 싶다면 BLOB나 LONGBLOB로 데이터 형식을 지정해야 함.

데이터 형식		바이트 수
TEXT 형식	TEXT	1~65535
	LONGTEXT	1~4294967295
BLOB 형식	BLOB	1~65535
	LONGBLOB	1~4294967295

날짜형

- 날짜 및 시간을 저장할 때 사용.

데이터 형식	바이트 수	설명
DATE	3	날짜만 저장. YYYY-MM-DD 형식으로 사용
TIME	3	시간만 저장. HH:MM:SS 형식으로 사용
DATETIME	8	날짜 및 시간을 저장. YYYY-MM-DD HH:MM:SS 형식으로 사용

제약조건

❖제약조건 (Constraint)

- 데이터의 무결성을 지키기 위해 데이터 입력 시 실행되는 검사 규칙

제약조건	설명
PRIMARY KEY	기본키로 설정된 속성에는 값이 중복될 수 없으며 NULL 값을 허용하지 않음
FOREIGN KEY	참조하는 테이블에 존재하는 값만 입력 가능하며 참조하는 속성은 반드시 기본키 또는 고유키여야 함
UNIQUE	중복되지 않는 유일한 값을 입력해야 하며 NULL 값을 허용함
CHECK	설정한 조건식을 만족한 데이터만 입력 가능
NOT NULL	생략 시 NULL 값을 허용 NOT NULL 설정 시 NULL 값을 허용하지 않음
DEFAULT	값을 입력하지 않았을 때 자동으로 입력될 값을 미리 지정

* 데이터의 무결성 : 데이터에 결함이 없음

외래키 제약조건 옵션

❖CASCADE

- 부모 테이블의 튜플이 변경될 때 자식 테이블의 튜플을 자동으로 변경
  - ON DELETE CASCADE: 부모 테이블의 튜플이 삭제될 때,
    해당 튜플을 참조하는 자식 테이블의 모든 튜플도 함께 삭제.
  - ON UPDATE CASCADE: 부모 테이블의 기본 키가 업데이트될 때,
    해당 키를 참조하는 자식 테이블의 외래 키 값도 함께 업데이트.

❖SET NULL

- 부모 테이블의 튜플이 변경될 때 자식 테이블의 외래 키 값을 NULL로 설정
  - ON DELETE SET NULL: 부모 테이블의 튜플이 삭제될 때,
    해당 레코드를 참조하는 자식 테이블의 외래 키 값이 NULL로 설정.
  - ON UPDATE SET NULL: 부모 테이블의 기본 키가 업데이트될 때,
    해당 키를 참조하는 자식 테이블의 외래 키 값이 NULL로 설정.

외래키 제약조건 옵션

❖RESTRICT

- 부모 테이블의 튜플이 변경될 때 자식 테이블의 튜플이 참조하고 있으면 변경을 거부
  - ON DELETE RESTRICT: 부모테이블의튜플이자식테이블에서참조되고있으면삭제거부
  - ON UPDATE RESTRICT: 부모테이블의튜플이자식테이블에서참조되고있으면업데이트 거부

❖NO ACTIOIN

- 부모 테이블의 튜플이 변경될 때 자식 테이블에 아무런 자동 변경을 수행하지 않음
- 부모 테이블의 변경이 자식 테이블의 제약 조건을 위반하면 에러를 발생
  - ON DELETE NO ACTION: 부모 테이블의 튜플이 삭제될 때
    자식 테이블에서 참조되고 있으면 에러를 발생시키고 삭제를 수행하지 않음
  - ON UPDATE NO ACTION: 부모 테이블의 기본 키가 업데이트될 때
    자식 테이블에서 참조되고 있으면 에러를 발생시키고 업데이트를 수행하지 않음

CREATE

❖CREATE 문

- 테이블을 구성하고, 속성과 속성에 관한 제약을 정의하며, 기본키 및 외래키를 정의하는 명령어.

```
CREATE TABLE 테이블이름
    ( {속성이름 데이터타입
        [NULL | NOT NULL | UNIQUE | DEFAULT 기본값 | CHECK 체크조건]
        }
        [PRIMARY KEY 속성이름(들)]
        [FOREIGN KEY 속성이름 REFERENCES 테이블이름(속성이름)]
            [ON DELETE {CASCADE | SET NULL}]
    )
```

CREATE

질의 3-34

다음과 같은 속성을 가진 NewBook 테이블을 생성하시오. 정수형은 INTEGER를 사용하며 문자형은 가변형 문자 타입인 VARCHAR을 사용한다.

- bookid(도서번호) – INTEGER
- bookname(도서이름) – VARCHAR(20)
- publisher(출판사) – VARCHAR(20)
- price(가격) – INTEGER

```
CREATE TABLE     NewBook (
   bookid        INTEGER,
   bookname      VARCHAR(20),
   publisher     VARCHAR(20),
   price         INTEGER);
```

CREATE

- 작성한 NewBook 테이블에는 아무 제약사항이 없음. 만약 기본키를 지정하고 싶다면 다음과 같이 생성함

```
CREATE TABLE     NewBook (
   bookid        INTEGER,
   bookname      VARCHAR(20),
   publisher     VARCHAR(20),
   price         INTEGER,
   PRIMARY KEY   (bookid));
```

- 앞의 CREATE 문은 다음과 같이 써도 같은 의미가 됨

```
CREATE TABLE     NewBook (
   bookid        INTEGER          PRIMARY KEY,
   bookname      VARCHAR(20),
   publisher     VARCHAR(20),
   price         INTEGER);
```

CREATE

질의 3-35

다음과 같은 속성을 가진 NewCustomer 테이블을 생성하시오.

- custid(고객번호) – INTEGER, 기본키
- name(이름) – VARCHAR(40)
- address(주소) – VARCHAR(40)
- phone(전화번호) – VARCHAR(30)

```
CREATE TABLE  NewCustomer (
   custid     INTEGER    PRIMARY KEY,
   name       VARCHAR(40),
   address    VARCHAR(40),
   phone      VARCHAR(30));
```

CREATE

질의 3-36

다음과 같은 속성을 가진 NewOrders 테이블을 생성하시오.

- orderid(주문번호) – INTEGER, 기본키
- custid(고객번호) – INTEGER, NOT NULL 제약조건, 외래키(NewCustomer.custid, 연쇄 삭제)
- bookid(도서번호) – INTEGER, NOT NULL 제약조건
- saleprice(판매가격) – INTEGER
- orderdate(판매일자) – DATE

```
CREATE TABLE  NewOrders (
   orderid    INTEGER,
   custid     INTEGER     NOT NULL,
   bookid     INTEGER     NOT NULL,
   saleprice  INTEGER,
   orderdate  DATE,
   PRIMARY KEY(orderid),
   FOREIGN KEY(custid) REFERENCES NewCustomer(custid) ON DELETE CASCADE);
```

ALTER

❖ALTER 문

- 생성된 테이블의 속성과 속성에 관한 제약을 변경하며, 기본키 및 외래키를 변경함

```
ALTER TABLE 테이블이름
   [ADD 속성이름 데이터타입]
   [DROP COLUMN 속성이름]
   [ALTER COLUMN 속성이름 데이터타입]
   [ALTER COLUMN 속성이름 [NULL | NOT NULL]]
   [ADD PRIMARY KEY(속성이름)]
   [[ADD | DROP] 제약이름]
```

ALTER

- 다음은 앞에서 실습한 NewBook 테이블을 생성하는 CREATE 문

```
CREATE TABLE   NewBook (
   bookid      INTEGER,
   bookname    VARCHAR(20),
   publisher   VARCHAR(20),
   price       INTEGER);
```

질의 3-37

NewBook 테이블에 VARCHAR(13)의 자료형을 가진 isbn 속성을 추가하시오.

```
ALTER TABLE NewBook ADD isbn VARCHAR(13);
```

ALTER

- 속성을 삭제하거나 제약사항을 변경하는 예

질의 3-38

NewBook 테이블에서 isbn 속성의 데이터 타입을 INTEGER형으로 변경하시오.

```
ALTER TABLE NewBook MODIFY isbn INTEGER;
```

질의 3-39

NewBook 테이블의 isbn 속성을 삭제하시오.

```
ALTER TABLE NewBook DROP COLUMN isbn;
```

ALTER

질의 3-40

NewBook 테이블의 bookname 속성에 NOT NULL 제약조건을 적용하시오.

```
ALTER TABLE NewBook MODIFY bookname VARCHAR(20) NOT NULL;
```

질의 3-41

NewBook 테이블의 bookid 속성을 기본키로 변경하시오.

```
ALTER TABLE NewBook ADD PRIMARY KEY(bookid);
```

DROP

❖DROP 문

- 테이블을 삭제하는 명령으로, 테이블의 구조와 데이터를 모두 삭제하므로 사용 시 주의
- 데이터만 삭제하려면 DELETE 문을 사용

```
DROP TABLE 테이블이름
```

- 예시:

질의 3-42

NewBook 테이블을 삭제하시오.

```
DROP TABLE NewBook;
```

DROP

질의 3-43

NewCustomer 테이블을 삭제하시오. 만약 삭제가 거절된다면 원인을 파악하고 관련된 테이블을 같이 삭제하시오(NewOrders 테이블이 NewCustomer를 참조하고 있다).

```
DROP TABLE NewCustomer;
```

- 삭제하려는 테이블의 기본키를 다른 테이블에서 참조하고 있다면 삭제가 거절됨
- NewCustomer 테이블을 삭제하기 위해서는 이 테이블을 참조하고 있는 NewOrders 테이블부터 삭제해야 함

```
DROP TABLE NewOrders;
DROP TABLE NewCustomer;
```

끝!